

DEVELOPMENT OF A MOBILE AUGMENTED REALITY APPLICATION AS A CONTROL INTERFACE FOR A ROBOT ARM

1st Muhammad Habib Abdullah
Department of Computer Engineering
Sepuluh Nopember Institute of
Technology
Surabaya, Indonesia
habibabdullahmuda@gmail.com

2nd Muhtadin, S.T., M.T.
Department of Computer Engineering
Sepuluh Nopember Institute of
Technology
Surabaya, Indonesia
muhtadin@its.ac.id

3rd Eko Pramunanto, S.T., M.T.
Department of Computer Engineering
Sepuluh Nopember Institute of
Technology
Surabaya, Indonesia
ekopram@its.ac.id

Abstract—The utilization of educational robots such as the Dobot Magician is becoming increasingly widespread. However, conventional control systems still present technical barriers such as a dependency on rigid numerical input and a lack of spatial visualization. This condition increases the risk of object manipulation failure or mechanical collisions when commands are executed directly on the physical robot. This research aims to design and implement a control system based on Android mobile devices by integrating Augmented Reality (AR) technology as an interactive visualization medium and the WebSocket protocol for real-time, two-way data communication. The system was developed using Unity with a marker-based tracking method supported by the Google ARCore SDK, mapping the virtual robot to a 1:1 scale using the Universal Robot Description Format (URDF). Results indicate successful, low-latency WebSocket communication and the physical robot's successful execution of sequential tasks via the Teach and Playback mode. The System Usability Scale (SUS) evaluation scored the application's functionality at 54.2 (Fair) and AR visualization quality at 70 (Good).

Index Terms—component, formatting, style, styling, insert

I. INTRODUCTION

Robots are increasingly utilized across various sectors, including industry, healthcare, and education, due to their ability to improve efficiency and minimize human error. Educational robotic arms like the Dobot Magician a 4 Degree of Freedom (4 DoF) manipulator are highly versatile, capable of picking and placing objects, drawing, and laser engraving.

Despite these advancements, typical human-robot interaction (HRI) relies heavily on rigid teach pendants or complex numerical coordinate inputs. The lack of a visual workspace representation prevents operators from easily validating trajectories, increasing the likelihood of mechanical collisions. To address these challenges, this study develops a mobile AR control interface that overlays a virtual representation of the robot onto the real world environment, allowing operators to simulate and adjust movements intuitively before executing them on the physical unit.

Identify applicable funding agency here. If none, delete this.

II. SYSTEM ARCHITECTURE AND METHODOLOGY

A. Hardware and Network Architecture

The system developed in this research is an augmented reality (AR)-based control system for a robot arm manipulator. The robot used is the Dobot Magician, which is controlled through an Android-based application. This system is designed to allow users to virtually simulate the robot's movements before they are executed by the physical robot.

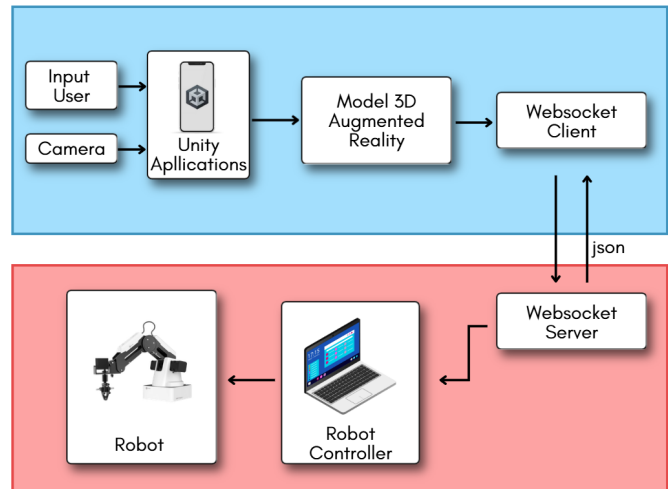


Fig. 1. Design Hardware

The AR application was developed using Unity to display a three-dimensional model of the robot in the real environment via a smartphone camera. Users can interactively adjust the position and movement of the virtual robot. Each movement made will be recorded as coordinate data, which is then sent to the control system.

The data transmission process is carried out using the WebSocket protocol from the Android device to a laptop acting as the server. The laptop then processes the data and translates it into motion commands using the PyDobot library.

Subsequently, the robot will execute the movements according to the data specified by the user.

This system applies the teach-and-playback method, where the user first determines a sequence of movement points (teaching), and then the robot executes those movements back in order (playback).

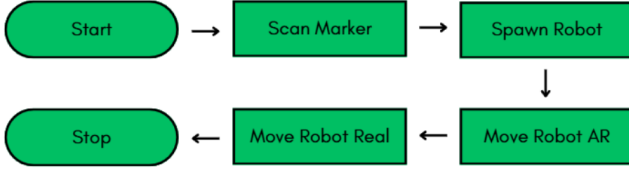


Fig. 2. Robot Control flow

Figure 2 illustrates the workflow for users to operate the Dobot robot through the application.

- 1) Start: The user launches the Mobile Augmented Reality (AR) control application on the smartphone, prompting the system to initialize the camera and establish the wireless network connection.
- 2) Scan Marker: The user points the smartphone camera to scan a physical marker or a flat workspace surface to define the reference coordinate point in the real-world space.
- 3) Spawn Robot: Upon successful scanning, the application automatically instantiates (spawns) a 3D model of the Dobot Magician robot as a 1:1 scale overlay on the smartphone screen.
- 4) Move Robot AR: The user provides motion instructions via the application interface. The virtual robot model responds instantly, and these joint angle changes are converted into a coordinate data packet.
- 5) Move Robot Real: The data packet from the phone is transmitted via the WebSocket protocol to the physical robot controller. The actual Dobot robot immediately moves to synchronize its position with the virtual model.
- 6) Stop: The control process concludes once the physical robot reaches the target position aligned with the virtual model, or when the user ends the application session.

B. AR Tracked Image

The implementation of Augmented Reality (AR) in this application utilizes XR Foundation technology in Unity through the AR Tracked Image Manager feature to detect physical markers in real-time. The process begins by registering the marker image into the Reference Image Library in Unity, complete with its actual size specifications in meters. This is done to ensure 1:1 scale accuracy of the 3D robot model when it is displayed in the real world. The marker also serves as a position reference for the virtual robot model to appear as if it exists in the real world through the phone screen.

When the application is running, the AR Session and XR Origin components are responsible for aligning real-world coordinates with the virtual space using position tracking

methods. When the phone camera captures a matching marker, a C# script with event handling functionality detects the Added condition to instantiate (spawn) the Dobot Magician virtual model prefab exactly on top of that marker.



Fig. 3. Marker

C. Hierarchical Structure of Robot 3D Virtual Models

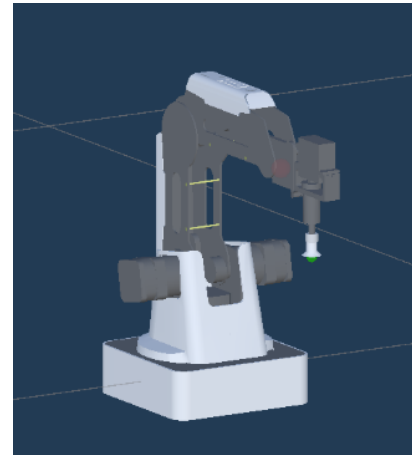


Fig. 4. Robot Model

The virtual Dobot Magician model used inside Unity originates from the robot's URDF (Unified Robot Description Format) file. This file contains the 3D geometry assets (mesh) along with the robot's physical constant data. The process of importing this URDF file into Unity yields the robot's constituent mesh components, whose initial structure remains unorganized.

Subsequently, a hierarchical reconstruction and setup of the parent-child relationship are performed so that the virtual robot model can be moved in accordance with the actual robot. The principle of this relationship dictates that movements applied to a parent object will automatically transform the position and rotation of the child objects underneath it; however, movements on a child object will not affect the parent object.

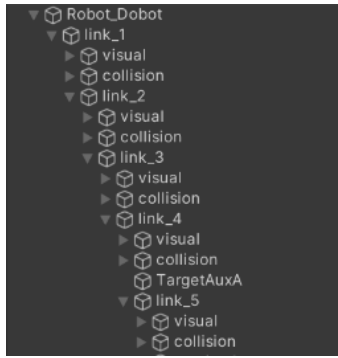


Fig. 5. Hierarchy Robot

Figure 5 show the structure of the robot model divided into 5 main components arranged into a single operational hierarchy as follows:

noitemsep

- 1) Static Base (Root Robot): Acts as the main parent that remains stationary and serves as the reference coordinate center (0,0,0).
- 2) Base Joint (Base Robot): Becomes a child of the static base, functioning as the actuator for the horizontal rotation of the entire robot body.
- 3) Shoulder Joint (Shoulder Joint): Becomes a child of the base joint, controlling the vertical orientation of the arm base.
- 4) Arm Joint (Forearm Joint): Becomes a child of the shoulder joint, regulating the vertical translation and forward-backward reach of the forearm.
- 5) Manipulator Tip (End-Effector): Becomes the final child at the end of the arm joint, serving as the final target position as well as the placement spot for the suction cup component.

In addition to the hierarchical configuration, manually adjusting the pivot points on each robot joint is also required. The default center coordinates of the raw 3D assets from the import process are generally located at the object's center of mass, rather than its actual mechanical axis. The pivot alignment is performed manually by placing the rotation center of the virtual object exactly at the physical axis point of the robot's motor. This is done so that each joint movement simulation in the application can produce a visualization identical to the movement of the actual Dobot robot.

D. Augmented Reality Display Performance

The design of the Augmented Reality (AR) interface on mobile devices is focused on mapping the system response logic to image marker tracking conditions and adjusting the user's viewport freedom. This system is designed to work adaptively through the AR Tracked Image Manager component in Unity and the Visual-Inertial Odometry (VIO) sensor from Google ARCore.



Fig. 6. Camera Distance

The Unity Augmented Reality application has limitations in detecting the marker. Even though we as users can see the marker through the phone camera, it does not necessarily mean the application detects it. As shown in Figure 6, there are two conditions with the same camera angle but different distances. The left image represents the condition where the marker is detected, and the right image represents the condition where the marker is not detected. The distance from the camera to the marker in the left image is approximately 30 cm, while in the right image, it is about 70 cm.

Even though the image marker is outside the application's detection range, the 3D virtual robot model is designed to still appear and remain stationary at the same real-world space coordinates. This self-tracking scheme utilizes the integration of the mobile device's internal gyroscope and accelerometer sensors through the Visual-Inertial Odometry (VIO) method. The calculated values from this sensor combination serve as the foundation for local transformations in dynamically defining the orientation, range distance, and dimensional scale of the virtual model following the user's phone movement.

E. User Interface (UI/UX)

When the application is first launched, the virtual Dobot Magician robot model is not yet displayed on the screen. The virtual model will only appear exactly on top of the marker after the user points the phone camera toward the designated marker. Before initiating control, users can perform a manual position calibration so that the virtual object can overlay precisely with the actual physical robot unit in the field.

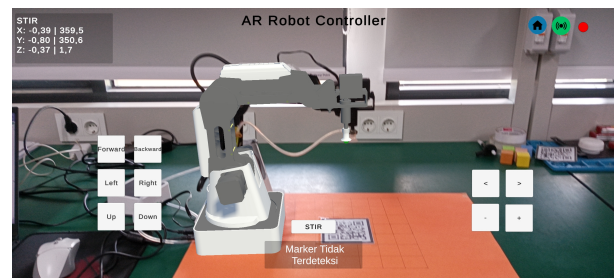


Fig. 7. Application interface after the robot model spawn process

Figure 7 shows the application interface view after the virtual robot model has been successfully spawned and its

position aligned with the actual robot. Upon initial initialization, the application automatically enters steering mode. This mode functions to adjust the spatial position of the virtual robot model, including translational shifts (right, left, forward, backward, up, down) as well as rotation (turning right or left).

The interface elements in this application are designed systematically with the following functional divisions: noitemsep

- 1) Left-Side Buttons: Function as directional controls to shift the position of the virtual robot model in three-dimensional space.
- 2) Right-Side Buttons: Consist of rotation adjustment buttons (right and left) to rotate the orientation of the virtual model, scale buttons (+ and −) to adjust the 3D model size, and function buttons to save coordinate data or robot movements.
- 3) Bottom-Center Buttons: Function as the main navigation menu to select the operational mode of the control system.
- 4) Top-Left Information: Displays the x, y, z position of the Dobot robot model and its rotation values.
- 5) Top-Right Circles: Indicators for the connection status with the robot controller, where green means connected and red means not connected.

F. Operational Modes

The application features multiple control paradigms:

- **Forward Kinematics (FK):** Allows incremental control of individual joints using interface buttons.
- **Inverse Kinematics (IK):** Enables users to determine a target point in 3D Cartesian space, prompting the system to calculate the necessary joint angles automatically.
- **Teach and Playback:** Allows users to record a sequence of target waypoints and transmit them as a bulk JSON payload to the robot for sequential execution.

G. WebSocket and JSON Communication

To ensure rapid and real-time data exchange between the Unity application on the smartphone as the client and the physical robot device as the server, this system utilizes the WebSocket communication protocol. Unlike the conventional HTTP protocol, which is request-response based, WebSocket provides a persistent, single-connection TCP (Transmission Control Protocol) full-duplex communication channel.

The data communication process initiates with a handshake stage over a local wireless network (Wi-Fi). The initialization workflow for this connection is designed through the following stages: noitemsep

- 1) Connection Request: The Unity application transmits a specific handshake request in the form of an HTTP Upgrade request to the specific IP address and port number allocated by the robot server gateway.
- 2) Validation and Persistent Connection: Once the server receives and approves the request, the protocol switches from HTTP to WebSocket.
- 3) Active Channel: The communication channel remains continuously open without the need to disconnect and

reconnect each time new data is transmitted. This minimizes functional overhead on the network data packets and reduces data transfer latency.

To enable the physical robot controller to process the waypoint queue data easily, all data is encapsulated into the JavaScript Object Notation (JSON) format. Based on the data processing script designed in Unity, the system loops to retrieve the joint angle parameters from each stored Waypoint object and aggregates them into a single text array.

III. TESTING

A. Testing Environment

The testing phase evaluates the functionality, accuracy, and performance of the augmented reality application and the designed WebSocket protocol. Experimental data collection was conducted inside the TW2-901 Computer Engineering Laboratory at the Sepuluh Nopember Institute of Technology to evaluate the control accuracy of the Android application.



Fig. 8. Side and Front View of the Environment

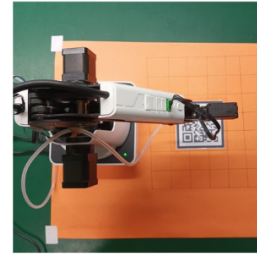


Fig. 9. Top View of the Environment

The environment shown in Figure 8 and Figure 9 features a flat laboratory workbench to prevent height variations across the workspace, with the marker affixed directly in front of the robot. Optimal marker detection requires the smartphone camera to face directly above the penanda at a distance of 20–30 cm. Once the 3D model spawns, the operator aligns it over the physical unit. The user then moves the smartphone to a distance of approximately 1 meter to achieve a broader field of view, establishing the ideal control positioning illustrated in Figure 10.

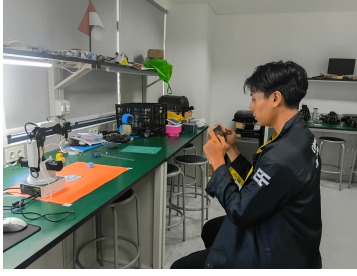


Fig. 10. Placement of the Robot and the User

This control system operates using two primary devices connected to the same local wireless Wi-Fi network: an Android smartphone serving as the AR client, and a laptop functioning as the gateway server to transmit instructions to the Dobot Magician. The technical specifications of the hardware are summarized in Table 4.1.

TABLE I
TESTING HARDWARE SPECIFICATIONS

No.	Hardware	Technical Specifications
1	Smartphone	Brand/Type: Infinix Note 12 2023 Operating System: Android 12 RAM Memory: 8 GB CPU: Helio G99 Rear Camera: 50 MP
2	Laptop	Brand/Type: Advan Pixelwar Processor: AMD Ryzen 5 6600H RAM Memory: 16 GB Operating System: Windows 11 Home 64-bit
3	Robot	Model: Dobot Magician Degree of Freedom: 4 DoF

B. Testing Scenarios

The evaluations are structured into three main phases. The first phase focuses on initial position calibration and joint angle synchronization accuracy between the virtual Unity model and the physical robot. The second phase validates operational modes, testing the Teach and Playback functionality and Move Box scenario via structured JSON data packets. The third phase comprises usability testing with human respondents to assess operational efficiency and execution time compared to conventional control software.

C. Control System and Augmented Reality Visualization Testing

These procedures measure application precision during physical manipulation and verify whether the sequential workflows generated within the interface translate accurately to mechanical motion.

1) *Virtual 3D Model Calibration and Synchronization Testing*: Calibration ensures spatial alignment by verifying that the coordinates and joint angles within the Unity Android environment match the physical Dobot Magician unit. Any spatial shift, however small, degrades tracking alignment and compromises visual overlay execution.

Testing involved powering the robot, executing a homing sequence, moving the arm to three distinct target blocks, and shutting down power. This sequence was repeated 5 times, recording the joint positions at each station. During this check, the system relied purely on interface positioning inputs while ignoring the virtual model overlay.

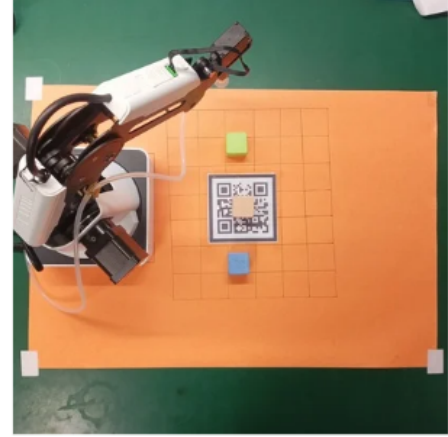


Fig. 11. Calibration Positions

TABLE II
DOBOT ROBOT JOINT ANGLE REPEATABILITY TEST RESULTS

Pengulangan	Posisi	Sendi 1	Sendi 2	Sendi 3
1	1	-17	51,83	66,24
2		-18,1	51,06	66,11
3		-19,29	51,08	66,63
4		-19,71	51,53	66,78
5		-19,94	51,4	66,44
1	2	-41,53	49,17	69,16
2		-42,19	48,75	68,59
3		-43,5	49,03	69,52
4		-44	48,66	68,75
5		-44,15	48,92	68,97
1	3	-63,26	50,74	66,44
2		-64,55	51,33	66,58
3		-65,48	51,32	67,3
4		-65,41	50,87	66,03
5		-65,83	51	66,22

Table II lists the recorded metrics from the calibration check. Following this, synchronization checks mapped positional uniformity between the physical arm and the virtual model. Identical targets were plotted in both systems across three unique points. The joint deviations were then measured at every position, utilizing the spatial setup illustrated in Figure 12.

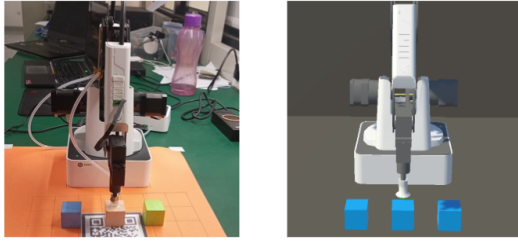


Fig. 12. Synchronization Position between the Dobot Robot and the Virtual Robot

2) *Teach and Playback Operational Mode Testing:* This phase evaluates physical joint manipulation across three block-displacement routines of varying complexity. The baseline task, Scenario 1, is mapped out in Figure 13.

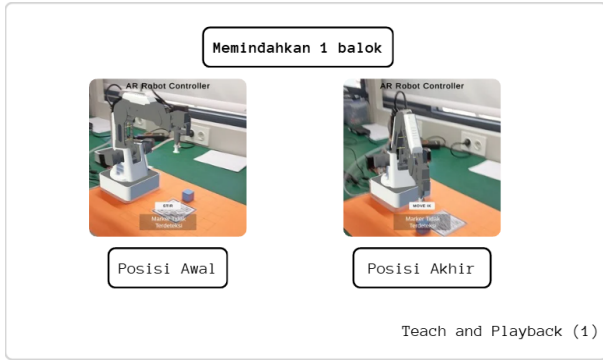


Fig. 13. Teach and Playback Position 1

In Scenario 1, the manipulator shifts a single block from the left side to the right side of the workspace. The user guides the virtual arm using teleoperation buttons, positioning the end-effector over the block and pressing "Add" to store the waypoint. Toggling the suction cup on creates an automated command marker. Once the load is lifted, the arm shifts to the destination point, where the user toggles the suction cup off. Pressing "Send" builds the sequential JSON packet and pushes it to the server.

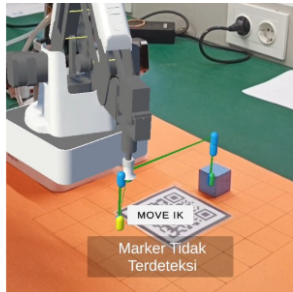


Fig. 14. Number of Waypoints for Scenario 1

Listing 4.1 shows the JSON package layout. Each entry holds variables J_1 through J_4 for the joint angles, coupled

with a boolean parameter to handle the suction cup state. This run requires a minimum of 8 waypoints to incorporate vertical clearance paths, preventing block collisions or surface dragging across the table.

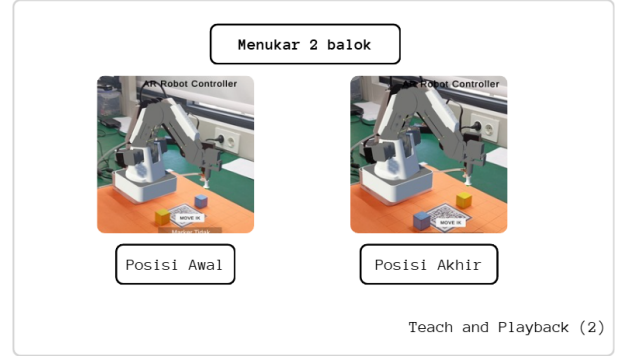


Fig. 15. Teach and Playback Position 2

Figure 15 highlights Scenario 2, which requires swapping two blocks. Initially, a blue block rests on the left and a yellow block rests on the right. The program concludes once their absolute workspace coordinates are inverted.

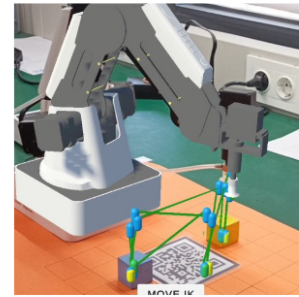


Fig. 16. Number of Waypoints for Scenario 2

Scenario 2 relies on 24 waypoints, as detailed in Listing 4.2. The longer path is a result of the step-by-step nature of the swap, which requires moving the first block to a temporary intermediate zone, placing the second block into the vacated starting node, and finally resetting the first block onto its final target position.

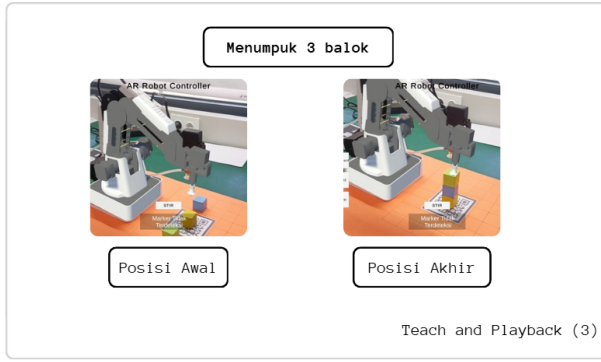


Fig. 17. Teach and Playback Position 3

Scenario 3 tests vertical stacking accuracy. As seen in Figure 17, the blocks begin at separate left, center, and right coordinates, and must be stacked vertically at the central base point.

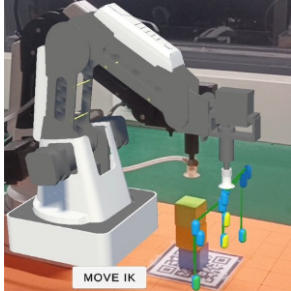


Fig. 18. Number of Waypoints for Scenario 3

This structure uses 16 waypoints (Listing 4.3). It requires fewer coordinates than the swap sequence because the center block remains static, meaning only the flanking components need to be moved. However, recording coordinates along the vertical Z-axis requires higher precision to ensure the structural stability of the stack when the suction cup disengages.

3) *Virtual Object Displacement Tracking (Move Box)*: Unlike the step-by-step recording mode, Move Box lets operators interact with the workspace dynamically. Instead of manual joint adjustments, users drag a virtual box across the screen, prompting the system to calculate the path automatically.



Fig. 19. Virtual Target Object Overlay

Figure 19 displays the interactive target models: a solid blue cube marks the initial load location, while a semi-transparent shell marks the target destination.

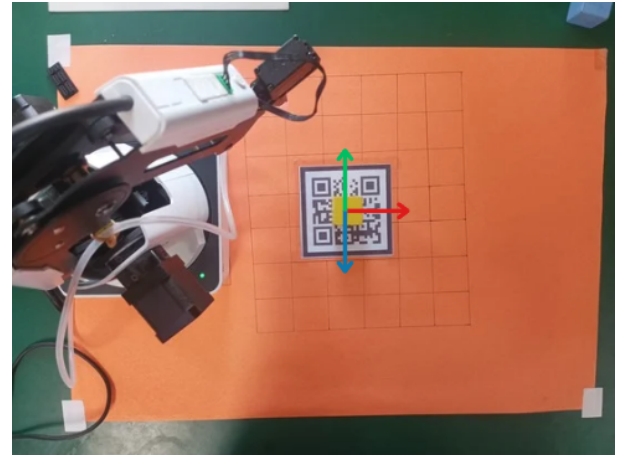


Fig. 20. Virtual Object Testing Targets

During this verification, a center box was moved sequentially across three coordinates (Figure 20), validating the inverse kinematics route generation without requiring manual waypoint entries.

IV. ANALYSIS AND DISCUSSION

This section analyzes the performance, calibration accuracy, and usability metrics collected during the testing phases.

A. Calibration and Virtual 3D Model Synchronization Analysis

As shown in Table III, measuring physical joint coordinates across 5 power-cycle iterations demonstrated high mechanical consistency. The analysis tracked average angular values alongside the maximum absolute variation across individual runs.

Joint 1 exhibited the largest absolute variation, consistently exceeding 2° , which is double the deviation observed in Joints 2 and 3. This variance stems from the difficulty of manually replicating horizontal coordinates compared to vertical paths.

TABLE III
AVERAGE VALUES AND MAXIMUM DEVIATIONS FROM CALIBRATION TESTING

	Sendi	Posisi 1	Posisi 2	Posisi 3
1	Rata-rata	-18,808	-43,074	-64,906
	Selisih	2,94	2,62	2,57
2	Rata-rata	51,38	48,906	51,052
	Selisih	0,77	0,51	0,59
3	Rata-rata	66,44	68,998	66,514
	Selisih	0,67	0,93	1,27

Standardizing these inputs requires direct numerical entry. Furthermore, neglecting the homing routine on Joint 1 led to positional drift.

Importantly, these deviations were caused by mechanical traits rather than network overhead or WebSocket instability. Because the deviations remained below 3° at the base and under 1° along the arm segments, they fall within safe tolerances, ensuring successful block manipulation matching the virtual display.

TABLE IV
PHYSICAL VS. VIRTUAL JOINT SYNCHRONIZATION LOGS

Posisi	Robot Dobot			Robot Virtual		
	Sendi 1	Sendi 2	Sendi 3	Sendi 1	Sendi 2	Sendi 3
1	-24,3	46,5	74,2	-17,3	42	64
2	-44,4	44,8	77,4	-35	40	65,3
3	-65,1	46,6	73,7	-52,7	40,8	63

Table IV compares the joint positions between the physical Dobot and the virtual representation across three equidistant target steps. The logs show consistent angular offsets between the two systems.

TABLE V
ABSOLUTE ANGULAR DIFFERENCES BETWEEN PHYSICAL AND VIRTUAL JOINTS

Posisi	Selisih		
	Sendi 1	Sendi 2	Sendi 3
1	7	4,5	10,2
2	9,4	4,8	12,1
3	12,4	5,8	10,7

Table V details these offsets across the test runs. While Joint 1 variations fluctuated across positions, Joints 2 and 3 deviations remained uniform. Joints 2 and 3 maintained tighter groupings than Joint 1, even though Joint 3 exhibited an absolute offset near 10° .

This fixed offset indicates a baseline shift between the internal coordinate mapping of the Dobot hardware and the local transform axes of the URDF model in Unity. The physical zero position is set by the internal homing routine, whereas the virtual zero position depends on the mesh pivot points defined during the rigging process.

Despite these reading offsets, the relationship between the two systems remained linear and synchronous. The physical

arm responded accurately and proportionally to coordinate changes sent from the AR interface.

B. Teach and Playback Operational Mode Analysis

TABLE VI
WAYPOINT PAYLOAD STATUS ACROSS TEST SCENARIOS

Skenario	Jumlah Waypoint	Panjang Karakter	Status
1	8	608	Berhasil
2	24	1822	Berhasil
3	16	1211	Berhasil

Table VI details the waypoint configurations compiled during testing. The waypoint count represents an optimized track layout rather than a rigid requirement. Operators can add or remove coordinates depending on the desired path. The optimal path uses multiples of 4 waypoints for pick-and-place movements: centering over the object, descending, toggling the suction cup, and returning to the clearance height.

Each waypoint generates approximately 72 characters. The largest payload occurred in Scenario 2, transmitting 1,822 characters to the server. The WebSocket connection handled all three scenarios successfully, resulting in accurate mechanical execution by the Dobot controller.

C. Virtual Object Displacement Tracking Analysis

TABLE VII
MOVE BOX COORDINATE EXECUTION OUTPUT

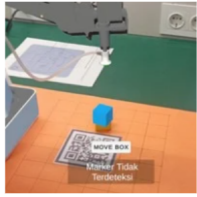
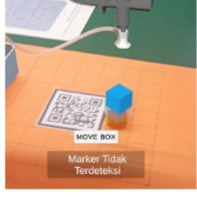
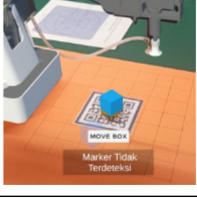
No	Posisi Awal	Posisi Akhir
1		
2		
3		

Table VII tracks the initial target coordinates and the final block placement achieved during Move Box mode testing. The final block placement matched the virtual boundaries closely. Minor rotational alignment errors occurred, but the Move Box

mode met its primary design goals by automatically translating virtual object manipulation into physical robot movement.

V. CONCLUSION

The mobile AR control interface successfully mitigates the reliance on rigid numerical inputs by providing a spatial, interactive visualization. While minor synchronization deviations exist between the physical and virtual joints, the Teach and Playback mechanism operates flawlessly. To overcome the ergonomic fatigue associated with mobile devices, future research should explore implementing this system on hands-free mixed-reality smart glasses.

REFERENCES

- [1] G. Singh and V. K. Banga, "Robots and its types for industrial applications," *Materials Today: Proceedings*, vol. 60, pp. 1779–1786, 2022.
- [2] Dobot, Dobot Magician User Guide. Shenzhen Yuejiang Technology Co., Ltd., 2023. Version 2.0.
- [3] T. B. Sheridan, "Human–robot interaction: Status and challenges," *Human Factors*, vol. 58, no. 4, pp. 525–532, 2016.
- [4] A. Prayudi, M. F. Arief, M. A. Zainuddin, S. Sukaridhoto, and H. Rante, "Wireless edge controller for robot education using mqtt on dobot magician lite," in *Digest of Technical Papers - IEEE International Conference on Consumer Electronics*, pp. 517–521, 2024.
- [5] H. Rante et al., "Integration of mqtt and augmented reality using dobot magician lite robot," *International Journal on Informatics Visualization*, vol. 9, no. 2, pp. 534–540, 2025.
- [6] H. Wang, X. Li, and X. Zhang, "Multimodal human-robot interaction on service robot," in *IEEE Advanced Information Technology, Electronic and Automation Control Conference (IAEAC)*, vol. 2021, pp. 2290–2295, 2021.
- [7] F. Muhammad, A. Hassan, A. Cleaver, and J. Sinapov, "Creating a shared reality with robots," in *ACM/IEEE International Conference on Human-Robot Interaction*, vol. 2019- March, pp. 614–615, 2019.
- [8] I. Fette and A. Melnikov, "The websocket protocol." <https://www.rfc-editor.org/info/rfc6455>, 2011.
- [9] Z. Makhataeva and H. A. Varol, "Augmented reality for robotics: A review," *Robotics*, vol. 9, no. 2, 2020.
- [10] K. Lotsaris, C. Gkournelos, N. Fousekis, N. Kousi, and S. Makris, "Ar based robot programming using teaching by demonstration techniques," *Procedia CIRP*, vol. 97, pp. 459–463, 2020.